

FUNCTION

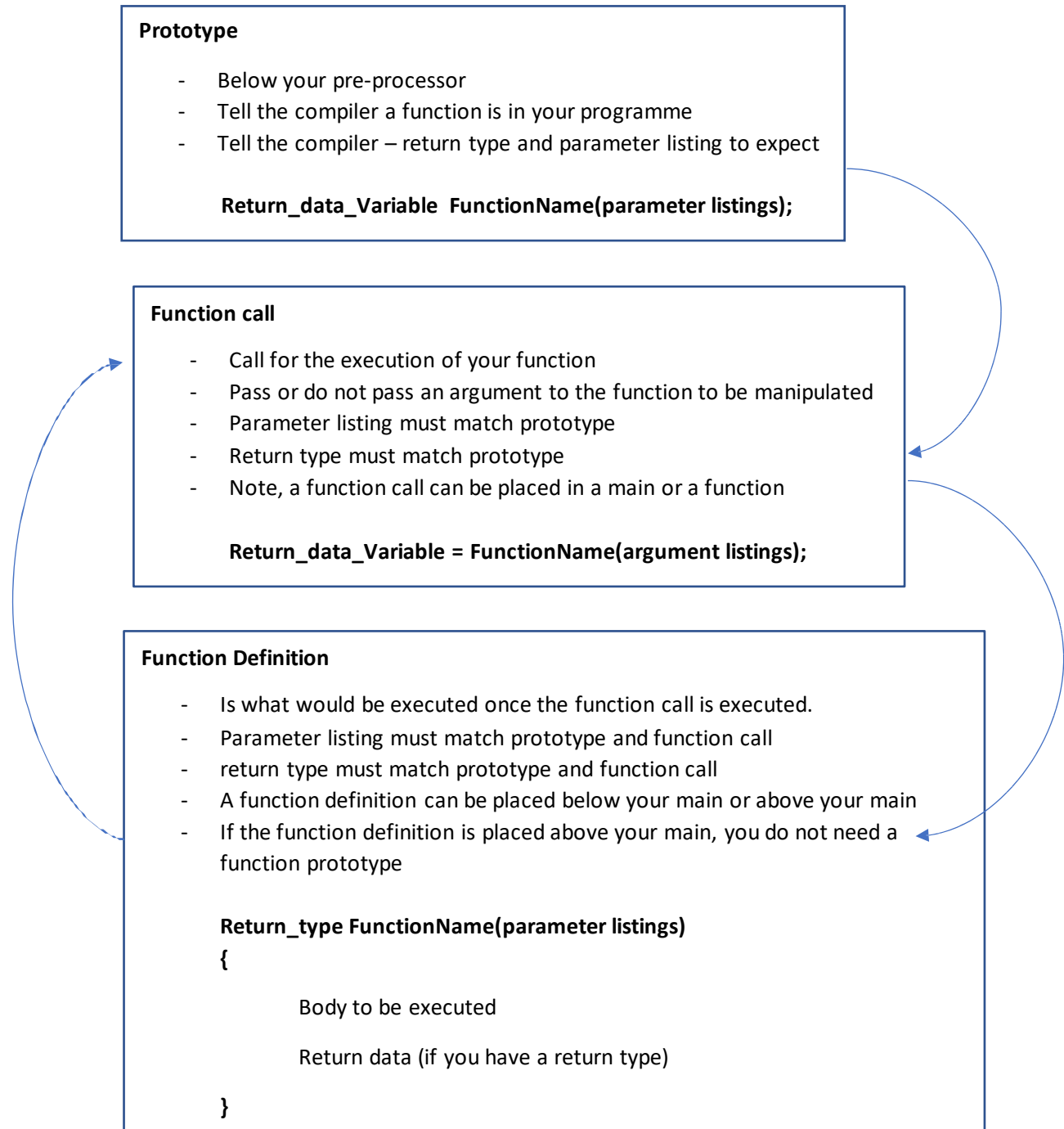
User-Defined Functions

Every component of a function must have a

- Return type
- Name
- Parameter/arguments

Three components to a function:

- Prototype
- Function Call
- Function Definition

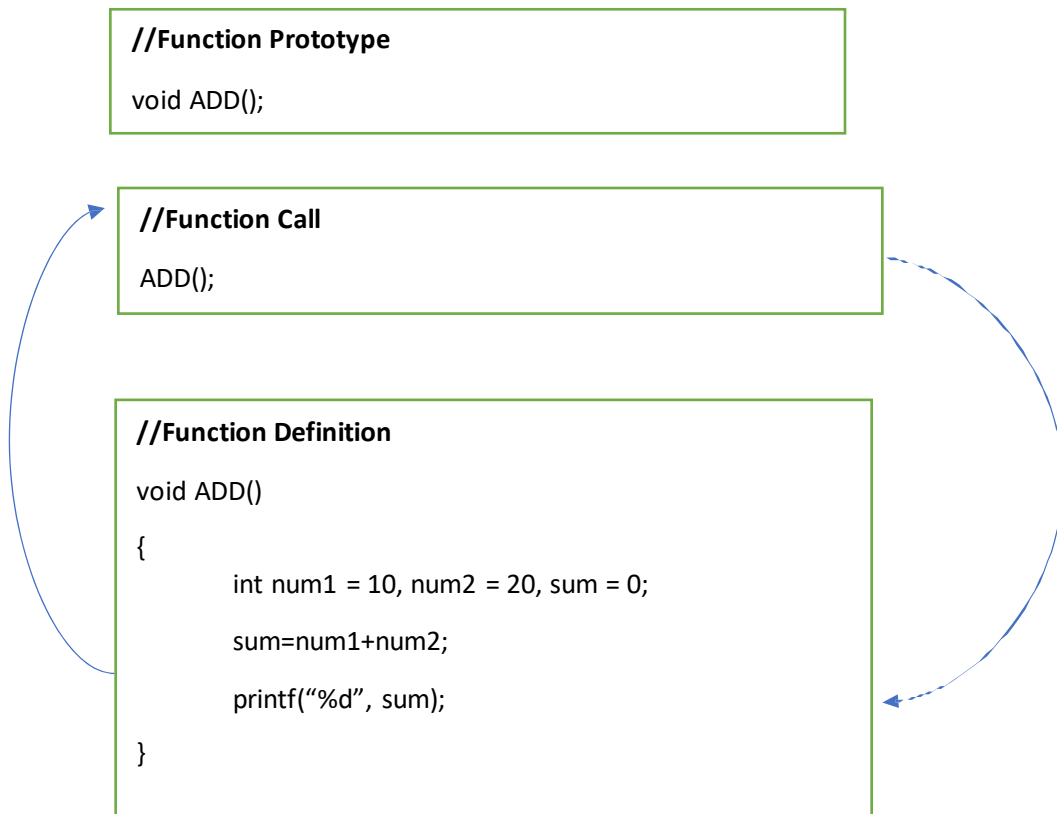


VARIABLES

- Global variables
 - o declared above Main
 - o can be seen and used by all functions
 - o do not need to be passed as an argument within a function call
- Local variables
 - o Declare within a function
 - o Cannot be seen or used by another function
 - o Must be passed as an argument from the function to another function to be used

TYPES OF FUNCTIONS

1. NO RETURN TYPE- NO PARAMETER LISTING



2. NO RETURN TYPE - PARAMETER LISTING

//Function Prototype

```
void ADD(int num1, int num2);
```

//Function Call

```
ADD(10, 20);
```

Or

```
int num1 =10, num2 =20;
```

```
ADD(num1, num2);
```

//Function Definition

```
void ADD(int num1, int num2)
{
    int sum = 0;
    sum = num1 + num2;
    printf("%d", sum);
}
```

10

20

3. RETURN RUN TYPE – NO PARAMETER LISTING

//Function Prototype

```
int ADD();
```

//Function Call

```
int sum=0;
```

```
Sum = ADD();
```

```
printf("%d",sum);
```

30

//Function Definition

```
int ADD()
```

```
{
```

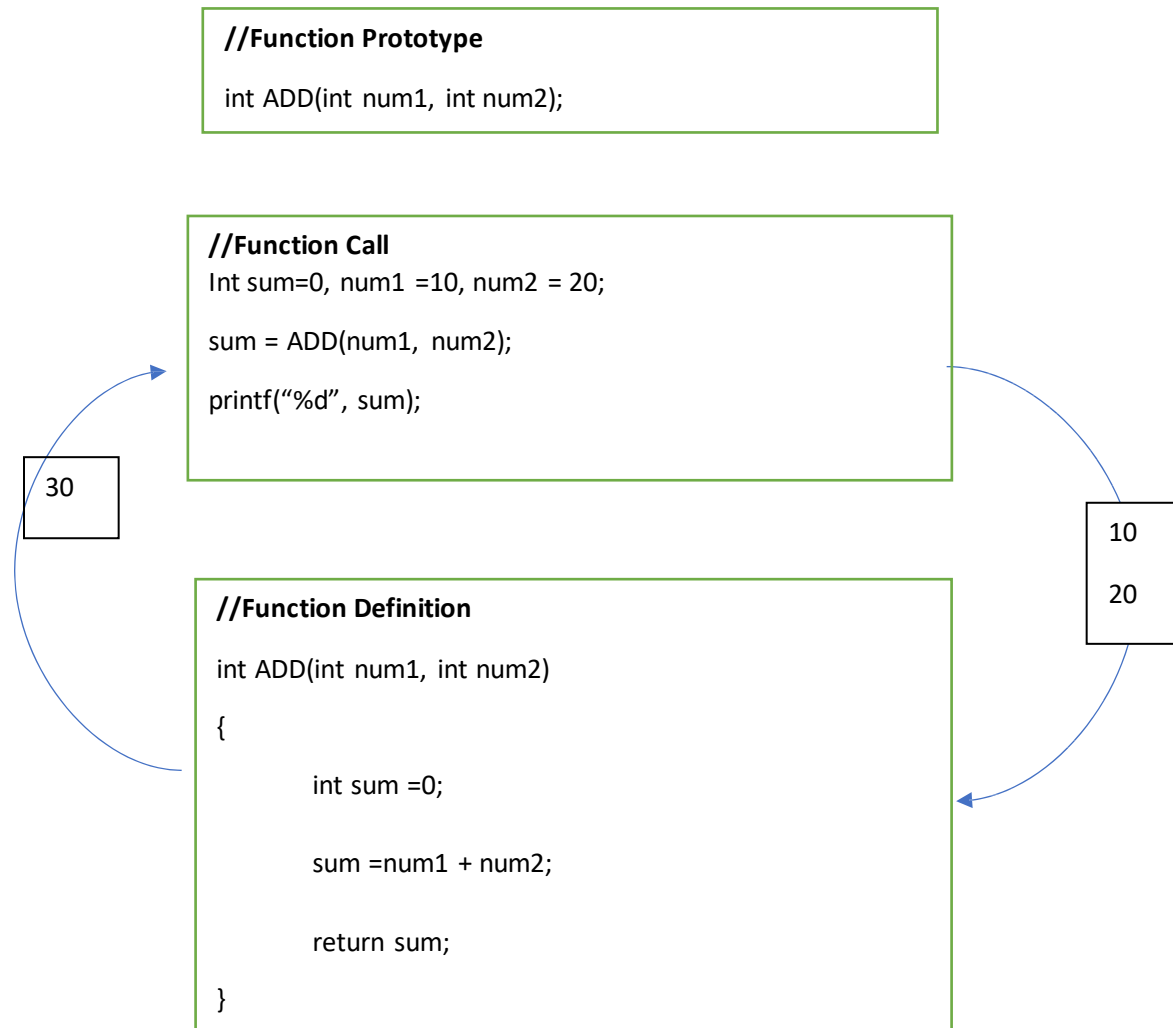
```
    int sum =0, num1 =10, num2 = 20;
```

```
    sum=num1 + num2;
```

```
    return sum;
```

```
}
```

4. RETURN RUN TYPE - PARAMETER LISTING



COMPLETE QUESTION

//C SOURCE CODE

```
#include <stdio.h>
```

//Function Prototype

```
int ADD(int, int );
```

//Main Function

```
int main()
```

```
{
```

```
    int sum;
```

//Function Call

```
    sum = ADD(10,20) ;
```

```
    printf("The sum is %d", sum);
```

```
    return 0;
```

```
}
```

//Function Definition

```
int ADD(int num1, int num2 )
```

```
{
```

```
    return num1 + num2;
```

```
}
```

//PYTHON SOURCE CODE

Function Definition

```
def add(num1, num2):
```

```
    return num1 + num2
```

Main logic

```
sum = add(10, 20)
```

```
print("The sum is", sum_value)
```

OR

```
print(f"The sum is {sum}")
```

Python does not need function prototypes because of how the language is executed and how names are resolved. Python executes code line by line, from top to bottom. A function is created and registered only when Python reaches its definition statement. Once the interpreter has seen the function definition, the function name exists and can be used. So as long as the function is defined before it is called, everything works. No advance declaration is required. Python does not require function prototypes because functions are defined at runtime, types are determined dynamically, and name resolution occurs during execution rather than compilation. In Python You do not declare parameter types. Since Python figures all of this at runtime, there is nothing a prototype needs to communicate.

Tutorial Questions

Please answer and discuss the questions below.

Part A

1. What is a function?
2. State two benefits of using functions in a program.
3. Differentiate between:
 - a. function definition
 - b. a function call
4. What is meant by function parameters and arguments?

Part B

Write programs in C and Python to answer the following questions.

1. Basic Function (No Parameters, No Return Value)

- a. Write a function named **printMessage** that prints "Welcome to UTech, the Birthplace of Greatness". Write the `main()` function or otherwise that calls `printMessage`.

2. Function with Parameters (No Return Value)

- a. Write a function named **printSum** that accepts two integers and prints their sum. Modify `main()` or otherwise to prompt the user for two numbers and pass them to `printSum`.

3. Function with Return Value

- a. Write a function named **square** that accepts an integer and returns its square. In `main()` or otherwise, call the function and display the returned value.

4. Multiple Parameters

- a. Write a function **calculateArea** that accepts the length and width of a rectangle and returns the area. Display the area in `main()` or otherwise.

5. Decision Making Inside a Function

- a. Write a function **isEven** that accepts an integer and returns 1 if the number is even and 0 otherwise. Use the function in `main()` or otherwise to display an appropriate message.

6. Iteration Inside a Function

- a. Write a function **printNumbers** that accepts an integer `n` and prints numbers from 1 to `n`. Call the function from `main()` or otherwise

Challenge Question

Write a function factorial that accepts a number and returns its factorial. Ensure the function handles the case where the user enters 0.

In the Laboratory

Complete any unfinished questions from the tutorial section. Implement and execute the following programs using C and Python:

1. Write a function that accepts a person's **name** and prints a greeting message.
2. Write a function that accepts **three test scores** and returns the **average score**.
3. Write a function that accepts a student's **score** and returns: "Pass" if score ≥ 50 or otherwise "Fail"
4. Write a function that accepts hours worked and hourly rate. The rules are:
 - a. Up to 40 hours $\rightarrow$ normal pay
 - b. More than 40 hours $\rightarrow$ extra hours paid at **1.5 $\times$ rate**

Return the total salary

5. Write a function that accepts the **number of GB used** and returns the bill:

Usage (GB)	Charge
Less than or equal to 5	\$2000
6 – 10	\$3500
More than 10	\$5000

6. Simple Payroll System Add any necessary input required (e.g. hours worked and hourly rate). Write functions to calculate the employee's gross pay, tax deduction, and net pay. Use these functions together to display an employee payslip.

Challenge Question

A utility company wants to calculate monthly electricity bills for its customers using a program that is structured with **functions**. The program must accept customer name and the number of electricity units consumed for the month.

Write the following functions:

1. **calculateEnergyCharge(units)**
 - First 100 units $\rightarrow$ \$4 per unit
 - Next 100 units $\rightarrow$ \$6 per unit
 - Any units above 200 $\rightarrow$ \$8 per unit
 - Return the energy charge

2. **calculateServiceCharge(energyCharge)**
 - Service charge is **5% of the energy charge**
 - Return the service charge
3. **calculateTotalBill(energyCharge, serviceCharge)**
 - Return the total bill
4. **displayBill(name, units, energyCharge, serviceCharge, totalBill)**
 - Display a **formatted electricity bill**

Program Requirements

- Use functions for all calculations
- Validate that units ≥ 0
- Process details for **multiple customers** using a loop